Session Review: Convolution, ReLU and Pooling

Feedback on a 2.5-hour learning session. Overall, five of six points were correct and one had a small formula slip. A fair score is about 90%, and the gaps are small details rather than misunderstandings.

1. Convolution as a dot product: correct (about 90%)

Convolution is the dot product between the filter's entries and the image patch under it, repeated at every position as the filter slides. A conv layer also adds a bias, so the full expression is `Σ x·w + b`. Technically it is cross-correlation, but everyone calls it convolution.

2. Why multiplying helps: correct

A dot product measures similarity, and a larger result means a stronger match at that location. Two details to add:

- Negative values mean the patch resembles the opposite pattern.
- The dot product isn't normalized, so a very bright patch can score high without being a perfect match. Values are best compared within the same feature map.

3. Feature map size: right idea, formula slip

The formula written was `in - k // stride + 1`, which Python reads as `in - (k // stride) + 1`. It needs parentheses:

`(in - k) // stride + 1`

For stride 1 and no padding, it simplifies to `in - k + 1`, so the answer happens to be right for that case. It applies to height and width separately, and the number of filters doesn't affect it.

4. Why ReLU: correct

Stacked linear layers collapse into one linear layer, so a non-linearity is needed to learn complex patterns. That is exactly the reason.

5. Output as channels of feature maps: correct, with one reorder

The number of channels equals the number of filters, which is right. The order is slightly off. The sequence is:

1. Dot product and sum
2. Add the bias
3. Only then apply ReLU

ReLU is a separate layer, not part of the convolution. Also, the sum runs over all input channels, so each filter produces **one** number per position, which is why F filters give F maps.

6. Pooling: correct

Each window produces one representative, so every region of the map is still represented, whether its values are large or near zero. To make it complete, add:

- Stride usually equals the window size, so windows don't overlap.
- It discards the exact position within the window.
- It has no learnable parameters.
- Its benefits are smaller maps, tolerance to small shifts, and a larger receptive field for the next layer.

Things from the session not in the summary

- **Stride in the code:** the first pooling version hardcoded the stride to the window size. It can now be separated from the window size.
- **Window vs. output size:** a window reads 4 values and gives 1, and the number of windows depends on the input.
- **A zero after ReLU** means "no strong match here", either unrelated or opposite, and ReLU can't tell which.

- **The backward pass:** pooling must remember which position won in each window, and ReLU remembers which values were positive.

Overall

The biggest improvement was in the code: from output-size and syntax bugs to a working pooling function, and noticing the stride issue independently.

Next session

1. Rewrite point 3 with the parentheses fixed, so the correct formula is what stays in the notes.
2. Add the backward pass to the pooling code.
3. Write flatten, dense, softmax, and cross-entropy.
4. Compare the layers with `cnn_scratch.py`.